

Chapter 14

Condition Number Estimation

*Most of LAPACK'S condition numbers and error bounds are based on
estimated condition numbers . . .*

*The price one pays for using an estimated
rather than an exact condition number is
occasional (but very rare) underestimates of the true error;
years of experience attest to the reliability of our estimators,
although examples where they badly underestimate the error can be constructed.*

— E. ANDERSON et al., *LAPACK Users' Guide, Release 2.0* (1995)

*The importance of the counter-examples is that they make clear that
any effort toward proving that the algorithms
always produce useful estimations is fruitless.*

*It may be possible to prove that the algorithms
produce useful estimations in certain situations, however,
and this should be pursued.*

An effort simply to construct more complex algorithms is dangerous.

— A. K. CLINE and R. K. REW, *A Set of Counter-Examples
to Three Condition Number Estimators* (1983)

Singularity is almost invariably a clue.

— SIR ARTHUR CONAN DOYLE, *The Boscombe Valley Mystery* "(1892)

14.1. How to Estimate Componentwise Condition Numbers

When bounding the forward error of a computed solution to a linear system we would like to obtain the bound with an order of magnitude less work than is required to compute the solution. For a dense $n \times n$ system, where the solution process usually requires $O(n^3)$ operations, we need to compute the bound in $O(n^2)$ operations. An estimate of the bound that is correct to within a factor 10 is usually acceptable, because it is the magnitude of the error that is of interest, not its precise value.

In the perturbation theory of Chapter 7 for linear equations we obtained perturbation bounds that involve the condition numbers

$$\kappa_{E,f}(A, x) = \frac{\|A^{-1}\| \|f\|}{\|x\|} + \|A^{-1}\| \|E\|,$$

$$\text{cond}_{E,f}(A, x) = \frac{\| |A^{-1}|f + |A^{-1}|E|x \|_{\infty}}{\|x\|_{\infty}},$$

and their variants. To compute these condition numbers exactly we need to compute A^{-1} , which requires $O(n^3)$ operations, even if we are given a factorization of A . Is it possible to produce reliable estimates of both condition numbers in $O(n^2)$ operations? The answer is yes, but to see why we first need to rewrite the expression for $\text{cond}_{E,f}(A, x)$. Consider the general expression $\| |A^{-1}|d \|_{\infty}$, where d is a given nonnegative vector (thus, $d = f + E|x|$ for $\text{cond}_{E,f}(A, x)$); note that the practical error bound (7.27) is of this form. Writing $D = \text{diag}(d)$ and $e = [1, 1, \dots, 1]^T$, we have

$$\| |A^{-1}|d \|_{\infty} = \| |A^{-1}|De \|_{\infty} = \| |A^{-1}|D|e \|_{\infty} = \| |A^{-1}|D \|_{\infty}. \quad (14.1)$$

Hence the problem is equivalent to that of estimating $\|B\|_{\infty} := \|A^{-1}D\|_{\infty}$. If we can estimate this quantity then we can estimate any of the condition numbers or perturbation bounds for a linear system. There is an algorithm, described in §14.3, that produces reliable order-of-magnitude estimates of $\|B\|_1$, for an arbitrary B , by computing just a few matrix-vector products Bx and $B^T y$ for carefully selected vectors x and y and then approximating $\|B\|_1 \approx \|Bx\|_1 / \|x\|_1$. If we assume that we have a factorization of A (say, $PA = LU$ or $A = QR$), then we can form the required matrix-vector products for $B = A^{-1}D$ in $O(n^2)$ flops. Since $\|B\|_1 = \|B^T\|_{\infty}$, it follows that we can use the algorithm to estimate $\| |A^{-1}|d \|_{\infty}$ in $O(n^2)$ flops.

Before presenting the 1-norm condition estimation algorithm, we describe a more general method that estimates $\|B\|_p$ for any $p \in [1, \infty]$. This more general method is of interest in its own right and provides insight into the special case $p = 1$.

14.2. The p -Norm Power Method

An iterative “power method” for computing $\|A\|_p$ was proposed by Boyd in 1974 [139, 1974]. When $p = 2$ it reduces to the usual power method applied to $A^T A$. We use the notation $\text{dual}_p(x)$ to denote any vector y of unit q -norm such that equality holds in the Holder inequality $x^T y \leq \|x\|_p \|y\|_q$ (this normalization is different from the one we used in (6.3), but is more convenient for our present purposes). Throughout this section, $p \geq 1$ and q is defined by $p^{-1} + q^{-1} = 1$.

Algorithm 14.1 (p -norm power method). Given $A \in \mathbb{R}^{m \times n}$ and $x_0 \in \mathbb{R}^n$, this algorithm computes γ and x such that $\gamma \leq \|A\|_p$ and $\|Ax\|_p = \gamma \|x\|_p$.

```

 $x = x_0 / \|x_0\|_p$ 
repeat
   $y = Ax$ 
   $z = A^T \text{dual}_p(y)$ 
  if  $\|z\|_q \leq z^T x$ 
     $\gamma = \|y\|_p$ 
    quit
end
 $x = \text{dual}_q(z)$ 
end

```

Cost: $4rmn$ flops (for r iterations).

There are several ways to derive Algorithm 14.1. Perhaps the most natural way is to examine the optimality conditions for maximization of

$$F(x) = \frac{\|Ax\|_p}{\|x\|_p}, \quad x \in \mathbb{R}^n.$$

First, we note that the subdifferential (that is, the set of subgradients) of an arbitrary vector norm $\|\cdot\|$ is given by [378, 1987, p. 379]

$$\partial\|x\| = \{ \lambda : \lambda^T x = \|x\|, \|\lambda\|_D \leq 1 \}.$$

If $x \neq 0$ then $\lambda^T x = \|x\| \Rightarrow \|\lambda\|_D \geq 1$, from the Holder inequality, and so, if $x \neq 0$,

$$\begin{aligned} \partial\|x\| &= \{ \lambda : \lambda^T x = \|x\|, \|\lambda\|_D = 1 \} \\ &=: \{\text{dual}(x)\}. \end{aligned}$$

It can also be shown that if A has full rank,

$$\partial\|Ax\| = \{A^T \text{dual}(Ax)\}. \quad (14.2)$$

We assume now that A has full rank, $1 < p < \infty$, and $x \neq 0$. Then it is easy to see that there is a unique vector $\text{dual}_p(x)$, so $\partial\|x\|_p$ has just one element, that is, $\|x\|_p$ is differentiable. Hence we have

$$F(x) = \frac{A^T \text{dual}_p(Ax)}{\|x\|_p} - \frac{\|Ax\|_p \text{dual}_p(x)}{\|x\|_p^2}. \quad (14.3)$$

The first-order Kuhn–Tucker condition for a local maximum of F is therefore

$$A^T \text{dual}_p(Ax) = \frac{\|Ax\|_p}{\|x\|_p} \text{dual}_p(x).$$

Since $\text{dual}_q(\text{dual}_p(x)) = x/\|x\|_p$ if $p \neq 1, \infty$, this equation can be written

$$x = \frac{\|x\|_p^2}{\|Ax\|_p} \text{dual}_q(A^T \text{dual}_p(Ax)). \quad (14.4)$$

The power method is simply functional iteration applied to this transformed set of Kuhn–Tucker equations (the scale factor $\|x\|_p^2/\|Ax\|_p$ is irrelevant since $F(ax) = F(x)$).

For the 1- and ∞ -norms, which are not everywhere differentiable, a different derivation can be given. The problem can be phrased as one of maximizing the convex function $F(x) := \|Ax\|_p$ over the convex set $S := \{x : \|x\|_p \leq 1\}$. The convexity of F and S ensures that, for any $u \in S$, at least one vector g exists such that

$$F(v) \geq F(u) + g^T(v - u) \quad \text{for all } v \in S. \quad (14.5)$$

Vectors g satisfying (14.5) are called *subgradients* of F (see, for example, [378, 1987, p. 364]). Inequality (14.5) suggests the strategy of choosing a subgradient g and moving from u to a point $v_* \in S$ that maximizes $g^T(v - u)$, that is, a vector that maximizes $g^T v$. Clearly, $v_* = \text{dual}_q(g)$. Since, from (14.2), g has the form $A^T \text{dual}_p(Ax)$, this completes the derivation of the iteration.

For all values of p the power method has the desirable property of generating an increasing sequence of norm approximations.

Lemma 14.2. *In Algorithm 14.1, the vectors from the k th iteration satisfy*

- (i) $z^{k^T} x^k = \|y^k\|_p$, and
- (ii) $\|y^k\|_p \leq \|z^k\|_q \leq \|y^{k+1}\|_p \leq \|A\|_p$.

The first inequality in (ii) is strict if convergence is not obtained on the k th iteration.

Proof. $z^{kT}x^k = \text{dual}_p(y^k)^T Ax^k = \text{dual}_p(y^k)^T y^k = \|y^k\|_p$. Then

$$\begin{aligned}\|y^k\|_p &= z^{kT}x^k \\ &\leq \|z^k\|_q \|x^k\|_p = \|z^k\|_q = z^{kT}x^{k+1} = \text{dual}_p(y^k)^T Ax^{k+1} \\ &\leq \|\text{dual}_p(y^k)\|_q \|Ax^{k+1}\|_p = \|y^{k+1}\|_p \\ &\leq \|A\|_p.\end{aligned}$$

For the last part, note that, in view of (i), the convergence test “ $\|z^k\|_q \leq z^{kT}x^k$ ” can be written as “ $\|z^k\|_q \leq \|y^k\|_p$ ”. $\square$

It is clear from Lemma 14.2 (or directly from the Holder inequality) that the convergence test “ $\|z\|_q \leq z^Tx$ ” in Algorithm 14.1 is equivalent to “ $\|z\|_q = z^Tx$ ” and, since $\|x\|_p = 1$, this is equivalent to $x = \text{dual}_q(z)$. Thus, although the convergence test compares two scalars, it is actually testing for equality in the vector equation (14.4).

The convergence properties of Algorithm 14.1 merit a careful description. First, in view of Lemma 14.2, the scalars $\gamma_k = \|y^k\|_p$ form an increasing and convergent sequence. This does not necessarily imply that Algorithm 14.1 converges, since the algorithm tests for convergence of the x_k , and these vectors could fail to converge. However, a subsequence of the x^k must converge to a limit, $\bar{x}$ say. Boyd [139, 1974] shows that if $\bar{x}$ is a strong local maximum of F with no zero components, then $x^k \rightarrow \bar{x}$ linearly.

If Algorithm 14.1 converges it converges to a stationary point of $F(x)$ when $1 < p < \infty$. Thus, instead of the desired global maximum $\|A\|_p$, we may obtain only a local maximum or even a saddle point. When $p = 1$ or ∞ , if the algorithm converges to a point at which F is not differentiable, that point need not even be a stationary point. On the other hand, for $p = 1$ or ∞ Algorithm 14.1 terminates in at most $n + 1$ iterations (assuming that when dual_p or dual_q is not unique an extreme point of the unit ball is taken), since the algorithm moves between the vertices e_i of the unit ball in the 1-norm, increasing F on each stage ($x = \pm e_i$ for $p = 1$, and $\text{dual}_p(y) = \pm e_i$ for $p = \infty$). An example where n iterations are required for $p = 1$ is given in Problem 14.2.

For two special types of matrix, more can be said about Algorithm 14.1.

- (1) If $A = xy^T$ (rank 1), the algorithm converges on the second step with $\gamma = \|A\|_p = \|x\|_p \|y\|_q$, whatever x_0 .
- (2) Boyd [139, 1974] shows that if A has nonnegative elements, $A^T A$ is irreducible, $1 < p < \infty$, and x_0 has positive elements, then the x^k converge and $\gamma_k \rightarrow \|A\|_p$.

For values of p strictly between 1 and ∞ the convergence behaviour of Algorithm 14.1 is typical of a linearly convergent method: exact convergence is not usually obtained in a finite number of steps and arbitrarily many steps can

be required for convergence, as is well-known for the 2-norm power method. Fortunately, there is a method for choosing a good starting vector that can be combined with Algorithm 14.1 to produce a reliable norm estimator; see the Notes and References and Problem 14.1.

We now turn our attention to the extreme values of p : 1 and ∞ .

14.3. LAPACK l-Norm Estimator

Algorithm 14.1 has two remarkable properties when $p = 1$: it almost always converges within four iterations (when $x_0 = [1, 1, \dots, 1]^T$, say) and it frequently yields $\|A\|_1$ exactly. This rapid, finite termination is also obtained for $p = \infty$, and is related to the fact that Algorithm 14.1 moves among the finite set of extreme points of the unit ball. Numerical experiments suggest that the accuracy is about the same for both norms but that slightly more iterations are required on average for $p = \infty$. Hence we will confine our attention to the 1-norm.

The 1-norm version of Algorithm 14.1 was derived independently of the general algorithm by Hager [492, 1984] and can be expressed as follows. The notation $\xi = \text{sign}(y)$ means that $\xi_i = 1$ or -1 according as $y_i \geq 0$ or $y_i < 0$. We now specialize to square matrices.

Algorithm 14.3 (1-norm power method). Given $A \in \mathbf{R}^{n \times n}$ this algorithm computes γ and x such that $\gamma \leq \|A\|_1$ and $\|Ax\|_1 = \gamma\|x\|_1$.

```

 $x = n^{-1}e$ 
repeat
   $y = Ax$ 
   $\xi = \text{sign}(y)$ 
   $z = A^T \xi$ 
  if  $\|z\|_\infty \leq z^T x$ 
     $\gamma = \|y\|_1$ 
    quit
  end
   $x = e_j$ , where  $|z_j| = \|z\|_\infty$  (smallest such  $j$ )
end
```

Numerical experiments show that the estimates produced by Algorithm 14.3 are frequently exact ($\gamma = \|A\|_1$), usually “acceptable” ($\gamma \geq \|A\|_1/10$), and sometimes poor ($\gamma < \|A\|_1/10$).

An important question for any norm or condition estimator is whether there exists a “counterexample”—a parametrized matrix for which the quotient “estimate divided by true norm” can be made arbitrarily small (or large, depending on whether the estimator produces a lower bound or an upper

bound) by varying a parameter. A general class of counterexample for Algorithm 14.3 is given by the matrices

$$A = I + \theta C,$$

where $Ce = C^T e = 0$ (there are many possible choices for C). For any such matrix, Algorithm 14.3 computes $y = n^{-1}e$, $\xi = e$, $z = e$, and hence the algorithm terminates at the end of the first iteration with

$$\frac{\gamma}{\|A\|_1} = \frac{1}{\|I + \theta C\|_1} \sim \theta^{-1} \quad \text{as } \theta \rightarrow \infty.$$

The problem is that the algorithm stops at a local maximum that can differ from the global one by an arbitrarily large factor.

A more reliable and more robust algorithm is produced by the following modifications of Higham [537, 1988].

Definition of estimate. To overcome most of the poor estimates, γ is redefined as

$$\gamma = \max \left\{ \|y\|_1, \frac{\|c\|_1}{\|b\|_1} \right\},$$

where

$$c = Ab, \quad b_i = (-1)^{i+1} \left(1 + \frac{i-1}{n-1} \right).$$

The vector b is considered likely to “pick out” any large elements of A in those cases where such elements fail to propagate through to y .

Convergence test. The algorithm is limited to a minimum of two and a maximum of five iterations. Further, convergence is declared after computing ξ if the new ξ is the same as the previous one; this event signals that convergence will be obtained on the current iteration and that the next (and final) multiplication $A^T \xi$ is unnecessary. Convergence is also declared if the new $\|y\|_1$ is no larger than the previous one. This nonincrease of the norm can happen only in finite precision arithmetic and signals the possibility of a vertex e_j being revisited—the onset of “cycling.”

The improved algorithm is as follows. This algorithm is the basis of all the condition number estimation in LAPACK.

Algorithm 14.4 (LAPACK norm estimator). Given $A \in \mathbb{R}^{n \times n}$ this algorithm computes γ and $v = Aw$ such that $\gamma \leq \|A\|_1$ with $\|v\|_1 / \|w\|_1 = \gamma$ (w is not returned).

$$\begin{aligned} v &= A(n^{-1}e) \\ \text{if } n &= 1, \text{ quit with } \gamma = |v_1|, \text{ end} \\ \gamma &= \|v\|_1 \\ \xi &= \text{sign}(v) \end{aligned}$$

```

 $x = A^T \xi$ 
 $k = 2$ 
repeat
   $j = \min\{i: |x_i| = \|x\|_\infty\}$ 
   $v = Ae_j$ 
   $\bar{\gamma} = \gamma$ 
   $\gamma = \|v\|_1$ 
  if  $\text{sign}(v) = \xi$  or  $\gamma \leq \bar{\gamma}$ , goto (*), end
   $\xi = \text{sign}(v)$ 
   $x = A^T \xi$ 
   $k = k + 1$ 
until  $(\|x\|_\infty = x_j \text{ or } k > 5)$ 
(*)  $x_i = (-1)^{i+1} \left(1 + \frac{i-1}{n-1}\right), \quad i = 1:n$ 
 $x = Ax$ 
if  $2\|x\|_1/(3n) > \gamma$  then
   $v = x$ 
   $\gamma = 2\|x\|_1/(3n)$ 
end

```

Algorithm 14.4 can still be “defeated”: it returns an estimate 1 for matrices $A(\theta)$ of the form

$$A(\theta) = I + \theta P, \text{ where } P = P^T, Pe = 0, Pe_1 = 0, Pb = 0. \quad (14.6)$$

(P can be constructed as $I - Q$ where Q is the orthogonal projection onto $\text{span}\{e, e_1, b\}$.) Indeed, the existence of counterexamples is intuitively obvious since Algorithm 14.4 samples the behaviour of A on less than n vectors in $\mathbb{R}^n$. Numerical counterexamples (not parametrized) can be constructed automatically by direct search, as described in §24.3.1. Despite these weaknesses, practical experience with Algorithm 14.4 shows that it is very rare for the estimate to be more than three times smaller than the actual norm, independent of the dimension n . Therefore Algorithm 14.4 is, in practice, a very *reliable* norm estimator. The number of matrix-vector products required is at least 4 and at most 11, and averages between 4 and 5.

There is an analogue of Algorithm 14.3 for complex matrices, in which ξ_i is defined as $y_i/|y_i|$ if $y_i \neq 0$ and 1 otherwise. In the corresponding version of Algorithm 14.4 the test for repeated ξ vectors is removed, because ξ now has noninteger, complex components and so is unlikely to repeat.

It is interesting to look at a performance profile of Algorithm 14.4. A performance profile is a plot of some measure of the performance of an algorithm versus a problem parameter. In this case, the natural measure of performance is the underestimation ratio, $\gamma/\|A\|_1$. Figure 14.1 shows the performance profile for a 5×5 matrix $A(\theta)$ of the form (14.6), with P constructed as

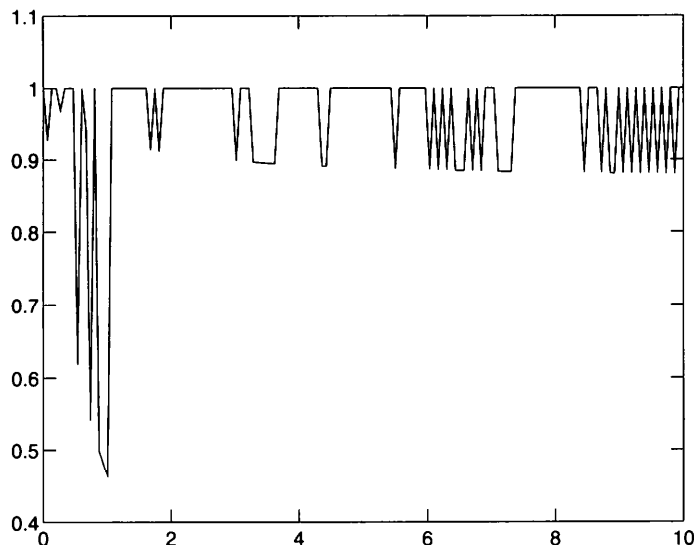


Figure 14.1. Underestimation ratio for Algorithm 14.4 for 5×5 matrix $A(\theta)$ of (14.6) with 150 equally spaced values of $\theta \in [0, 10]$.

described above (because of rounding errors in constructing $A(\theta)$ and within the algorithm, the computed norm estimates differ from those that would be produced inexact arithmetic). The jagged nature of the performance curve is typical for algorithms that contain logical tests and branches. Small changes in the parameter θ , which themselves result in different rounding errors, can cause the algorithm to visit different vertices in this example.

14.4. Other Condition Estimators

The first condition estimator to be widely used is the one employed in LINPACK. It was developed by Cline, Moler, Stewart, and Wilkinson [216, 1979]. The idea behind this condition estimator originates with Gragg and Stewart [476, 1976], who were interested in computing an approximate null vector rather than estimating the condition number itself.

We will describe the algorithm as it applies to a triangular matrix $T \in \mathbb{R}^{n \times n}$. There are three steps:

1. Choose a vector d such that $\|y\|$ is as large as possible relative to $\|d\|$, where $T^T y = d$.
2. Solve $Tx = y$.

3. Estimate $\|T^{-1}\| \approx \|x\|/\|y\|$ ($\leq \|T^{-1}\|$).

In LINPACK the norm is the 1-norm, but the algorithm can also be used for the 2-norm or the cm-norm. The motivation for step 2 is based on a singular value decomposition analysis. Roughly, if $\|y\|/\|d\|$ ($\approx \|T^{-T}\|$) is large then $\|x\|/\|y\|$ ($\approx \|T^{-1}\|$) will almost certainly be at least as large, and it could be a much better estimate. Notice that $T^T T x = d$, so the algorithm is related to the power method on the matrix $(T^T T)^{-1}$ with the specially chosen starting vector d .

To examine step 1 more closely, suppose that $T = U^T$ is lower triangular and note that the equation $Uy = d$ can be solved by the following column-oriented (saxpy) form of substitution:

```

p(1:n) = 0
for j = n: -1: 1
    y_j = (d_j - p_j)/u_jj
(*)  p(1: j - 1) = p(1: j - 1) + U(1: j - 1, j)y(j)
end

```

The idea is to choose the elements of the right-hand side vector d adaptively as the solution proceeds, with $d_j = \pm 1$. At the j th stage of the algorithm $d_1, \dots, d_{j+1}$ have been chosen and $y_n, \dots, y_{j+1}$ are known. The next element $d_j \in \{+1, -1\}$ is chosen so as to maximize a weighted sum of $d_j - p_j$ and the partial sums $p_1, \dots, p_j$ which would be computed during the next execution of statement (*) above. Hence the algorithm looks ahead, trying to gauge the effect of the choice of d_j on future solution components. This heuristic algorithm for choosing d is expressed in detail as follows.

Algorithm 14.5 (LINPACK condition estimator). Given a nonsingular upper triangular matrix $U \in \mathbb{R}^{n \times n}$ and a set of nonnegative weights $\{w_i\}$, this algorithm computes a vector y such that $Uy = d$, where the elements $d_j = \pm 1$ are chosen to make $\|y\|$ large.

```

p(1:n) = 0
for j = n: -1: 1
    y_j^+ = (1 - p_j)/u_jj
    y_j^- = (-1 - p_j)/u_jj
    p^+(1: j - 1) = p(1: j - 1) + U(1: j - 1, j)y_j^+(j)
    p^-(1: j - 1) = p(1: j - 1) + U(1: j - 1, j)y_j^-(j)
    if w_j|1 - p_j| + \sum_{i=1}^{j-1} w_i|p_i^+| \geq w_j|1 + p_j| + \sum_{i=1}^{j-1} w_i|p_i^-|
        y_j = y_j^+
        p(1: j - 1) = p^+(1: j - 1)
    else
        y_j = y_j^-

```

```

    p(1:j-1) = p'(1:j-1)
end
end

```

cost: $4n^2$ flops.

LINPACK takes the weights $w_j \equiv 1$, though another possible (but more expensive) choice would be $w_j = 1/|u_{jj}|$, which corresponds to how p_j is weighted in the expression $y_j = (d_j - p_j)/u_{jj}$.

To estimate $\|A^{-1}\|$ for a full matrix A , the LINPACK estimator makes use of an LU factorization of A . Given $PA = LU$, the equations solved are $U^T z = d$, $L^T y = z$, and $AX = P^T y$, where for the first system d is constructed by the analogue of Algorithm 14.5 for lower triangular matrices; the estimate is $\|x\|_1/\|y\|_1 \approx \|A^{-1}\|_1$. Since d is chosen without reference to L , there is an underlying assumption that any ill condition in A is reflected in U . This assumption may not be true; see Problem 14.3.

In contrast to the LAPACK norm estimator, the LINPACK estimator requires explicit access to the elements of the matrix. Hence the estimator cannot be used to estimate componentwise condition numbers. Furthermore, separate code has to be written for each different type of matrix and factorization. Consequently, while LAPACK has just a single norm estimation routine, which is called by many other routines, LINPACK has multiple versions of its algorithm, each tailored to the specific matrix or factorization.

Several years after the LINPACK condition estimator was developed, several parametrized counterexamples were found by Cline and Rew [217, 1983]. Numerical counterexamples can also be constructed by direct search, as shown in §24.3.1. Despite the existence of these counterexamples the LINPACK estimator has been widely used and is regarded as being almost certain to produce an estimate correct to within a factor 10 in practice.

A 2-norm condition estimator was developed by Cline, Corm, and Van Loan [218, 1982, Algorithm 1]; see also Van Loan [1043, 1987] for another explanation. The algorithm builds on the ideas underlying the LINPACK estimator by using “look-behind” as well as look-ahead. It estimates $\sigma_{\min}(R) = \|R^{-1}\|_2^{-1}$ or $\sigma_{\max}(R) = \|R\|_2$ for a triangular matrix R , where $\sigma_{\min}$ and $\sigma_{\max}$ denote the smallest and largest singular values, respectively. Full matrices can be treated if a factorization $A = QR$ is available (Q orthogonal, R upper triangular), since R and A have the same singular values. The estimator performs extremely well in numerical tests, often producing an estimate that has some correct digits [218, 1982], [534, 1987]. No counterexamples to the estimator were known until Bischof [103, 1990] obtained counterexamples as a by-product of the analysis of a different but related method, mentioned at the end of this section.

All the methods described so far have the property that when applied repeatedly to a given matrix they always produce the same estimate. Another

approach is to introduce some randomness, so that the output of the method depends on the particular random numbers chosen. A natural idea along these lines is to apply the power method to the matrix $(AA^T)^{-1}$ with a randomly chosen starting vector. If a factorization of A is available, the power method vectors can be computed inexpensively by solving linear systems with A and AT . Analysis based on the singular value decomposition suggests that there is a high probability that a good estimate of $\|A^{-1}\|_2$ will be obtained. This notion is made precise by Dixon [306, 1983], who proves the following result.

Theorem 14.6 (Dixon). *Let $A \in \mathbb{R}^{n \times n}$ be nonsingular and let $\theta > 1$ be a constant. If $x \in \mathbb{R}^n$ is a random vector from the uniform distribution on the unit sphere $S_n = \{y \in \mathbb{R}^n : y^T y = 1\}$, then the inequality*

$$(x^T (AA^T)^{-k} x)^{1/2k} \leq \|A^{-1}\|_2 \leq \theta (x^T (AA^T)^{-k} x)^{1/2k} \quad (14.7)$$

holds with probability at least $1 - 0.8\theta^{-k/2} n^{1/2}$ ($k \geq 1$). $\square$

Note that the left-hand inequality in (14.7) always holds; it is only the right-hand inequality that is in question.

For $k = 1$, (14.7) can be written as

$$\|A^{-1}x\|_2 \leq \|A^{-1}\|_2 \leq \theta \|A^{-1}x\|_2,$$

which suggests the simple estimate $\|A^{-1}\|_2 \approx \|A^{-1}x\|_2$, where x is chosen randomly from the uniform distribution on S_n . Such vectors x can be generated from the formula

$$x_i = z_i / \|z\|_2,$$

where $z_1, \dots, z_n$ are independent random variables from the normal $N(0, 1)$ distribution [668, 1981, p. 130]. If, for example, $n = 100$ and θ has the rather large value 6400 then inequality (14.7) holds with probability at least 0.9.

In order to take a smaller constant θ , for fixed n and a desired probability, we can use larger values of k . If $k = 2j$ is even then we can simplify (14.7), obtaining

$$\|(AA^T)^{-j} x\|_2^{1/2j} \leq \|A^{-1}\|_2 \leq \theta \|(AA^T)^{-j} x\|_2^{1/2j}, \quad (14.8)$$

and the minimum probability stated by the theorem is $1 - 0.8\theta^{-j} n^{1/2}$. Taking $j = 3$, for the same value $n = 100$ as before, we find that (14.8) holds with probability at least 0.9 for the considerably smaller value $\theta = 4.31$.

Probabilistic condition estimation has not yet been adopted in any major software packages, perhaps because the other techniques work so well. For more on the probabilistic power method approach see Dixon [306, 1983], Higham [534, 1987], and Kuczynski and Wozniakowski [676, 1992] (who also analyse the more powerful Lanczos method with a random starting vector). For a probabilistic condition estimation method of very general applicability

see Kenney and Laub [652, 1994] and Gudmundsson, Kenney, and Laub [485, 1995].

The condition estimators described above assume that a single estimate is required for a matrix given in its entirety. Condition estimators have also been developed for more specialized situations. Bischof [103, 1990] develops a method for estimating the smallest singular value of a triangular matrix which processes the matrix a row or a column at a time. This “incremental condition estimation” method can be used to monitor the condition of a triangular matrix as it is generated, and so is useful in the context of matrix factorization such as the QR factorization with column pivoting. The estimator is generalized to sparse matrices by Bischof, Lewis, and Pierce [104, 1990]. Barlow and Vemulapati [67, 1992] develop a 1-norm incremental condition estimator with look-ahead for sparse matrices.

Condition estimates are also required in applications where a matrix factorization is repeatedly updated as a matrix undergoes low rank changes. Algorithms designed for a recursive least squares problem and employing the Lanczos method are described by Ferng, Golub, and Plemmons [372, 1991]. Pierce and Plemmons [831, 1992] describe an algorithm for use with the Cholesky factorization as the factorization is updated, while Shroff and Bischof [918, 1992] treat the QR factorization.

14.5. Condition Numbers of Tridiagonal Matrices

For a bidiagonal matrix B , $|B^{-1}| = M(B)^{-1}$ (see §8.3), so the condition numbers $\kappa_{E,f}$ and $\text{cond}_{E,f}$ can be computed exactly with an order of magnitude less work than is required to compute B^{-1} explicitly. This property holds more generally for several types of tridiagonal matrix, as a consequence of the following result. Recall that the LU factors of a tridiagonal matrix are bidiagonal and may be computed using the formulae (9.16).

Theorem 14.7. *If the nonsingular tridiagonal matrix $A \in \mathbb{R}^{n \times n}$ has the LU factorization $A = LU$ and $|L||U| = |A|$, then $|U^{-1}||L^{-1}| = |A^{-1}|$.*

Proof. Using the notation of (9.15), $|L||U| = |A| = |LU|$ if and only if, for all 2,

$$|l_i e_{i-1} + u_i| = |l_i| |e_{i-1}| + |u_i|,$$

that is, if

$$\text{sign} \left(\frac{l_i e_{i-1}}{u_i} \right) = 1. \quad (14.9)$$

Using the formulae

$$(U^{-1})_{ij} = \frac{1}{u_j} \prod_{p=i}^{j-1} \left(\frac{-e_p}{u_p} \right), \quad j \geq i,$$

$$(L^{-1})_{ij} = \prod_{p=j}^{i-1} (-l_{p+1}), \quad i \geq j,$$

we have

$$\begin{aligned} (U^{-1}L^{-1})_{ij} &= \sum_{k=\max(i,j)}^n (U^{-1})_{ik} (L^{-1})_{kj} \\ &= \sum_{k=\max(i,j)}^n \frac{1}{u_k} \prod_{p=i}^{k-1} \left(\frac{-e_p}{u_p} \right) \prod_{p=j}^{k-1} (-l_{p+1}) \\ &= \prod_{p=i}^{\max(i,j)-1} \left(\frac{-e_p}{u_p} \right) \cdot \prod_{p=j}^{\max(i,j)-1} (-l_{p+1}) \\ &\quad \times \sum_{k=\max(i,j)}^n \frac{1}{u_k} \prod_{p=\max(i,j)}^{k-1} \left(\frac{e_p l_{p+1}}{u_p} \right) \\ &= \prod_{p=i}^{\max(i,j)-1} \left(\frac{-e_p}{u_p} \right) \cdot \prod_{p=j}^{\max(i,j)-1} (-l_{p+1}) \\ &\quad \times \frac{1}{u_{\max(i,j)}} \cdot \sum_{k=\max(i,j)}^n \prod_{p=\max(i,j)}^{k-1} \left(\frac{e_p l_{p+1}}{u_{p+1}} \right). \end{aligned}$$

Thus, in view of (14.9), it is clear that $|U^{-1}L^{-1}|_{ij} = (|U^{-1}||L^{-1}|)_{zj}$, as required. $\square$

Since L and U are bidiagonal, $|U^{-1}| = M(U)^{-1}$ and $|L^{-1}| = M(L)^{-1}$. Hence, if $|A| = |L||U|$, then, from Theorem 14.7,

$$|A^{-1}|y = |U^{-1}||L^{-1}|y = M(U)^{-1}M(L)^{-1}y. \quad (14.10)$$

It follows that we can compute any of the condition numbers or forward error bounds of interest *exactly* by solving two bidiagonal systems. The cost is $O(n)$ flops, as opposed to the $O(n^2)$ flops needed to compute the inverse of a tridiagonal matrix.

When does the condition $|A| = |L||U|$ hold? Theorem 9.11 shows that it holds if the tridiagonal matrix A is symmetric positive definite, totally positive, or an M-matrix. So for these types of matrix we have a very satisfactory way to compute the condition number.

If A is tridiagonal and diagonally dominant by rows, then we can compute in $O(n)$ flops an upper bound for the condition number that is not more than a factor $2n - 1$ too large.

Theorem 14.8. *Suppose the nonsingular, row diagonally dominant tridiagonal matrix $A \in \mathbb{R}^{n \times n}$ has the LU factorization $A = LU$. Then, if $y \geq 0$,*

$$\| |U^{-1}| |L^{-1}| y \|_{\infty} \leq (2n-1) \| |A^{-1}| y \|_{\infty}.$$

Proof. We have $L^{-1} = UA^{-1}$, so

$$|U^{-1}| |L^{-1}| y \leq |U^{-1}| |U| |A^{-1}| y = |V^{-1}| |V| |A^{-1}| y,$$

where the bidiagonal matrix $V = \text{diag}(u_{ii})^{-1} U$ has $v_{ii} \equiv 1$ and $|v_{i,i+1}| = |e_i/u_i| \leq 1$ (see the proof of Theorem 9.12). Thus

$$|U^{-1}| |L^{-1}| y \leq \begin{bmatrix} 1 & 1 & \dots & 1 \\ & 1 & \dots & 1 \\ & & \ddots & \vdots \\ & & & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & & \\ & 1 & \ddots & \\ & & \ddots & 1 \\ & & & 1 \end{bmatrix} |A^{-1}| y,$$

and the result follows on taking norms. $\square$

In fact, it is possible to compute $|A^{-1}|_y$ exactly in $O(n)$ operations for any tridiagonal matrix. This is a consequence of the special form of the inverse of a tridiagonal matrix.

Theorem 14.9 (Ikebe). *Let $A \in \mathbb{R}^{n \times n}$ be tridiagonal and irreducible (that is, $a_{i+1,i}$ and $a_{i,i+1}$ are nonzero for all i). Then there are vectors x , y , p , and q such that*

$$(A^{-1})_{ij} = \begin{cases} x_i y_j, & i \leq j, \\ p_i q_j, & i \geq j. \end{cases} \quad \square$$

This result says that the inverse of an irreducible tridiagonal matrix is the upper triangular part of a rank-1 matrix joined along the diagonal to the lower triangular part of another rank-1 matrix. If A is reducible then it has the block form $\begin{bmatrix} A_{11} & 0 \\ A_{21} & A_{22} \end{bmatrix}$ (or its transpose), and this blocking can be applied recursively until the diagonal blocks are all irreducible, at which point the theorem can be applied to the diagonal blocks.

The vectors x , y , p , and q in Theorem 14.9 can all be computed in $O(n)$ flops, and this enables the condition numbers and forward error bounds to be computed also in $O(n)$ flops (see Problem 14.5). Unfortunately, the vectors x , y , p , and q can have a huge dynamic range, causing the computation to break down because of overflow or underflow. For example, for the diagonally dominant tridiagonal matrix with $a_{ii} \equiv 4$, $a_{i+1,i} = a_{i,i+1} \equiv 1$, we have ($x_1 = 1$) $|x_n| \approx \theta^{n-1}$, $|y_1| \approx \theta^{-1}$, and $|y_n| \approx \theta^{-n}$, where $\theta = 2 + \sqrt{3} \approx 3.73$. These numerical problems can be overcome, but only at a nontrivial increase in cost. Therefore, we do not recommend the use of Theorem 14.9 for computing condition numbers. For a general tridiagonal matrix it is probably better to *estimate* the condition number using Algorithm 14.4.